Yoonie Operating System

Web × staff app: flows, endpoints, operating rules and reference

Part A · Flows and APIs

1 Introduction

1.1 Overview

Yoonie runs a hotel from booking to checkout. A guest books on the hotel's own site; Yoonie plans the work that booking creates; staff are dispatched on their phones and report each step from where the work happens; the desk watches it live on the web and answers only when something goes wrong. Every step is a structured, timestamped event, ready for analytics and AI.

Part A describes each flow: an overview with its sequence diagram, the endpoints the web and app call, and each endpoint's request and response. Part B is the reference: data schemas, roles, decisions, open items, build status and source documents. The rules every flow follows are in `docs/OPERATING_RULES.md`; this document shows how they run.

1.2 Parties

Party	Role
Guest	Books rooms, transfers, tours and spa on the hotel's site. Pays at the hotel today.
Booking site	Public web at `/` on the hotel's domain. Holds inventory, then creates the order.
Ops web	Staff dashboard at `ops.<domain>`: Rooms, Guests, Services, Dispatch, Property. Login required.
Staff app	Expo app on staff phones. One home screen per job: Housekeeper, Driver, Dispatcher, Manager; plus Inbox.
Yoonie API	NestJS under `/api`. Owns inventory, orders, dispatch tasks, rooms, notifications and audit.
Planner	Inside the API. Turns each booking into tasks (room turnaround, airport pickup, tour fill check).
Notification worker	Runs every minute. Delivers pending notifications by Expo push; everything also lands in the inbox.
MongoDB Atlas	One database for all branches. Every document carries `branchId`.
LINE / WhatsApp	Guest and staff messaging channels. LINE linking is built; sending to LINE is not wired yet.

1.3 Flows

Section	Flow	Main endpoints	Status
2	Guest booking	POST /inventory/hold, POST /orders	Built (pay at hotel)
3	Dispatch planning	Planner (internal), GET /dispatch	Built
4	Room turnaround	POST /staff/rooms/{id}/transition	Built
5	Airport pickup	POST /staff/pickups/{id}/step, /issue	Built; app not shipped
6	Tour fill decision	POST /dispatch/{id}/decision	Built
7	Front desk money	POST /orders/{id}/payments	Built
8	Ops boards	GET /rooms/live-occupancy, /ops-board/*	Built
9	Notifications and inbox	Worker, /notifications, /devices	Built; LINE not wired
10	Staff app sign-in and offline	/auth/login, offline queue	Built
–	Tour departure, spa session	–	Designed, not built

1.4 Surfaces

Surface	Screen	What it's for
Booking site	/	Search dates, pick room, add transfer / tour / spa / dining, check out
Ops web	Rooms	Read-only state: board with ready-by countdowns, floor map, timeline, room sheet with folio
Ops web	Guests	Who is in house, arriving, departing; their services and balance
Ops web	Services	Capacity vs demand per slot; revenue against fixed cost
Ops web	Dispatch	The one inbox: today's runway, rooms / pickups / tours, assign, inspect, answer problems
Ops web	Property	Rooms, services, restaurants catalogue (admin)
Staff app	Housekeeper	Today's rooms (mine + open pool): Start, Mark clean
Staff app	Driver	Today's pickups: next step button, arrival sign, Report a problem
Staff app	Dispatcher	All tasks; answer pickup problems; Keep / Merge tours
Staff app	Manager	Inspect: Pass, or Fail with reason
Staff app	Inbox	Every notification; acknowledge critical ones

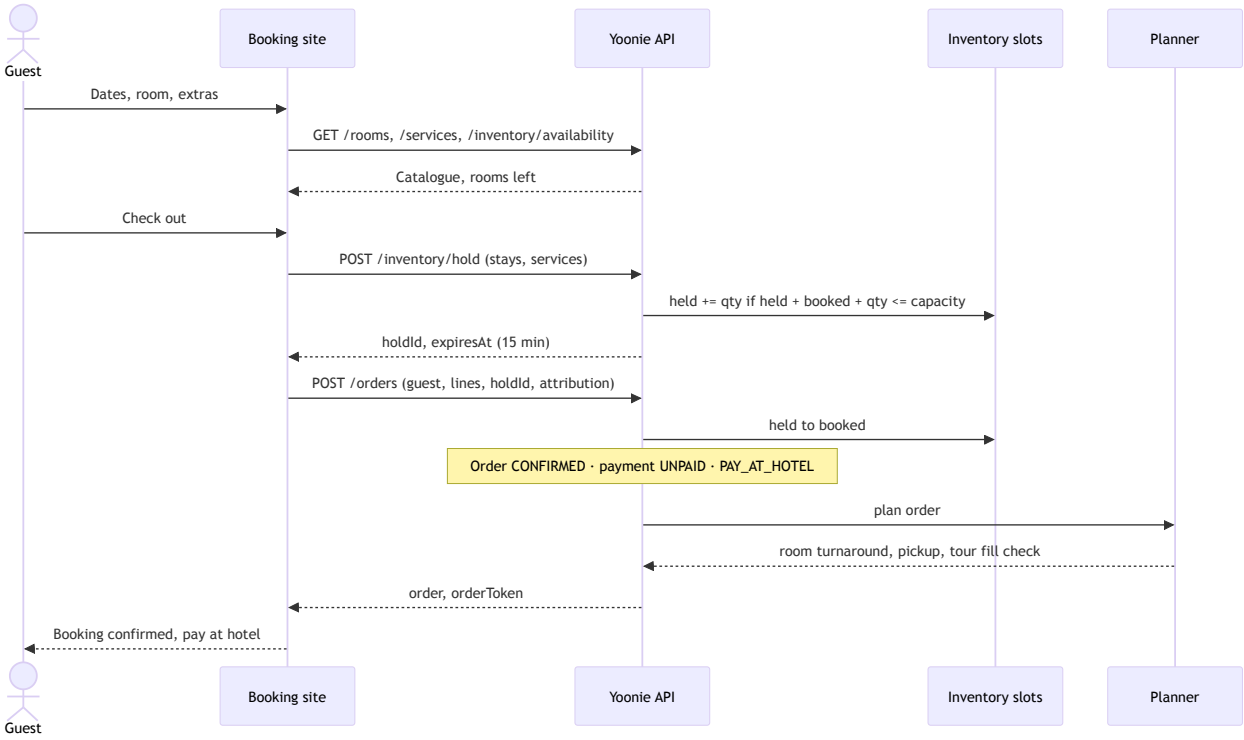
1.5 Yoonie API Conventions

Item	Value
Base path	`/api` (web proxies `/api` to `BACKEND_URL`)
Authentication	JWT bearer access token; refresh via `POST /auth/refresh`. Catalogue and checkout routes are public.
Branch scope	Staff act only within their `branchIds`; platform admins see all
Success body	`{ success: true, statusCode, data, timestamp }`
Error body	`{ success: false, statusCode, message, path, timestamp }`
Validation	Unknown fields are rejected (whitelist + forbidNonWhitelisted)
Offline safety	`Idempotency-Key` header on staff mutations; repeated steps are no-ops
Formats	Money in THB. Business dates `YYYY-MM-DD` next to UTC timestamps. Times shown in Asia/Bangkok.
Rate limit	100 requests per 60 seconds per IP (configurable)
Polling	Live occupancy every 10 s; Dispatch, roster and boards every 15 s

2 Guest Booking

2.1 Overview

The guest picks dates and a room, adds extras, and checks out. Yoonie holds the capacity for 15 minutes, creates the order as confirmed with payment at the hotel, and hands it to the planner.



2.2 Endpoints

Endpoint	Who	Result
GET /rooms?branchId	Public	Room types and rates
GET /services?branchId	Public	Transfers, tours, spa
GET /restaurants?branchId	Public	Dining venues
GET /inventory/availability	Public	Rooms left per type and range
POST /inventory/hold	Public	Hold ACTIVE, expires in 15 min
DELETE /inventory/hold/{id}	Public (hold token)	Hold RELEASED
POST /orders	Public	Order CONFIRMED, tasks planned
GET /orders/{id}	Staff (branch) or guest (order token)	Order with charges
POST /orders/{id}/lines/{lineId}/cancel	Staff or guest (order token)	Line CANCELLED, capacity returned

2.3 Hold Inventory

Method	Request URL	Who can call
POST	`/api/inventory/hold`	Public

Request Body

Property Name	Description	Data Type
`branchId`	Branch id	String
`cartId`	Checkout session id	String

Property Name	Description	Data Type
`stays[]`	roomType, checkInDate, checkOutDate, qty. Expanded into room-nights.	Array
`services[]`	serviceId, startsAt, pax. Transfer hour, tour departure or treatment start.	Array
`lines[]`	Raw slotKey + qty (advanced)	Array
`ttlMinutes`	Hold lifetime; default branch holdTtlMinutes (15)	Number

Example

```
{
  "branchId": "665a1b2c3d4e5f6a7b8c9d0e",
  "cartId": "cart_abc123",
  "stays": [
    {
      "roomType": "DELUXE_BALCONY",
      "checkInDate": "2026-10-25",
      "checkOutDate": "2026-10-27",
      "qty": 1
    }
  ],
  "services": [
    {
      "serviceId": "665a1b2c3d4e5f6a7b8c9d3c",
      "startsAt": "2026-10-25T06:20:00.000Z",
      "pax": 2
    }
  ]
}
```

Response Body

Property Name	Description	Data Type
`_id`	Hold id; send as holdId on the order	String
`status`	ACTIVE, CONVERTED, RELEASED, EXPIRED	String
`lines[]`	slotKey, qty actually held	Array
`expiresAt`	When the sweeper returns the capacity	DateTime
`holdToken`	Lets the guest release their own hold	String

If any slot is full the whole hold fails with **409** and nothing is held.

2.4 Create Order

Method	Request URL	Who can call
POST	`/api/orders`	Public

Request Body

Property Name	Description	Data Type
`branchId`	Branch id	String
`guest`	fullName, email, phoneE164, lineUserId, preferredLanguage (en / th)	Object
`lines[]`	kind STAY / DINING / SERVICE, title, unitPriceThb, startsAt, businessDate, stay / dining / service detail	Array
`lines[].service.flight`	Transfer only: number, landingAt, meetingDoor. Missing means "needs details".	Object
`holdId`	From 2.3; converted from held to booked	String
`attribution`	utmSource, utmMedium, utmCampaign, channel	Object
`specialRequests`	Free text	String

Example


```
{
  "branchId": "665a1b2c3d4e5f6a7b8c9d0e",
  "holdId": "665a1b2c3d4e5f6a7b8c9d5e",
  "guest": {
    "fullName": "Hiroshi Tanaka",
    "email": "hiroshi@example.com",
    "phoneE164": "+81901234567",
    "preferredLanguage": "en"
  },
  "lines": [
    {
      "kind": "STAY",
      "title": "Deluxe Balcony (2 nights)",
      "unitPriceThb": 7600,
      "startsAt": "2026-10-25T07:00:00.000Z",
      "businessDate": "2026-10-25",
      "stay": {
        "roomType": "DELUXE_BALCONY",
        "checkInDate": "2026-10-25",
        "checkOutDate": "2026-10-27"
      }
    },
    {
      "kind": "SERVICE",
      "title": "Airport transfer",
      "unitPriceThb": 1200,
      "startsAt": "2026-10-25T06:20:00.000Z",
      "businessDate": "2026-10-25",
      "service": {
        "serviceKind": "AIRPORT_TRANSFER",
        "pax": 2,
        "flight": {
          "number": "EK 372",
          "landingAt": "2026-10-25T06:20:00.000Z",
          "meetingDoor": 4
        }
      }
    }
  ],
  "attribution": {
    "utmSource": "instagram",
    "channel": "DIRECT_WEB"
  }
}
```

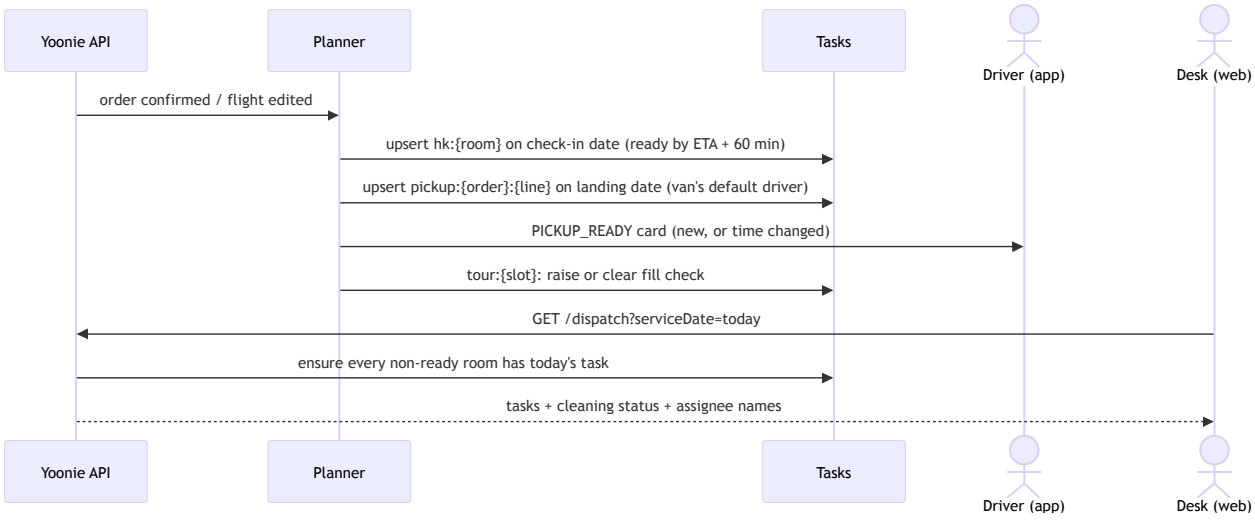

Response Body

Property Name	Description	Data Type
`order.orderNumber`	e.g. TS-261025-A0EAA	String
`order.status`	CONFIRMED (pay at hotel)	String
`order.payment`	method PAY_AT_HOTEL, status UNPAID, entries[]	Object
`order.totals`	subtotal, service charge 10%, VAT 7%, grand total	Object
`orderToken`	Signed token; lets the guest view or cancel without an account	String

3 Dispatch Planning

3.1 Overview

Every booking becomes work. The planner turns order lines into tasks with an owner, a deadline and a natural key, so planning twice never creates a duplicate. Loading today's Dispatch also tops up room work for every room that isn't ready.



3.2 Task Types

Type	Natural key	Created when	Deadline	Who does it
HOUSEKEEPING_TURNAROUND	hk:{roomNumber}	Booking; or Dispatch load for any non-ready room	Flight landing (or check-in time) + 60 min	Housekeeper (open pool)
AIRPORT_PICKUP	pickup:{orderId}:{lineId}	Transfer booked with flight; flight edited	Flight landing	Driver
TOUR_CONSOLIDATION	tour:{slotKey}	Departure below minimum pax or under 40% full	Before departure	Dispatcher decides

3.3 Endpoints

Method	Request URL	Who can call
GET	`/api/dispatch?branchId&serviceDate&status&type`	Admin, Manager, Dispatcher, Staff

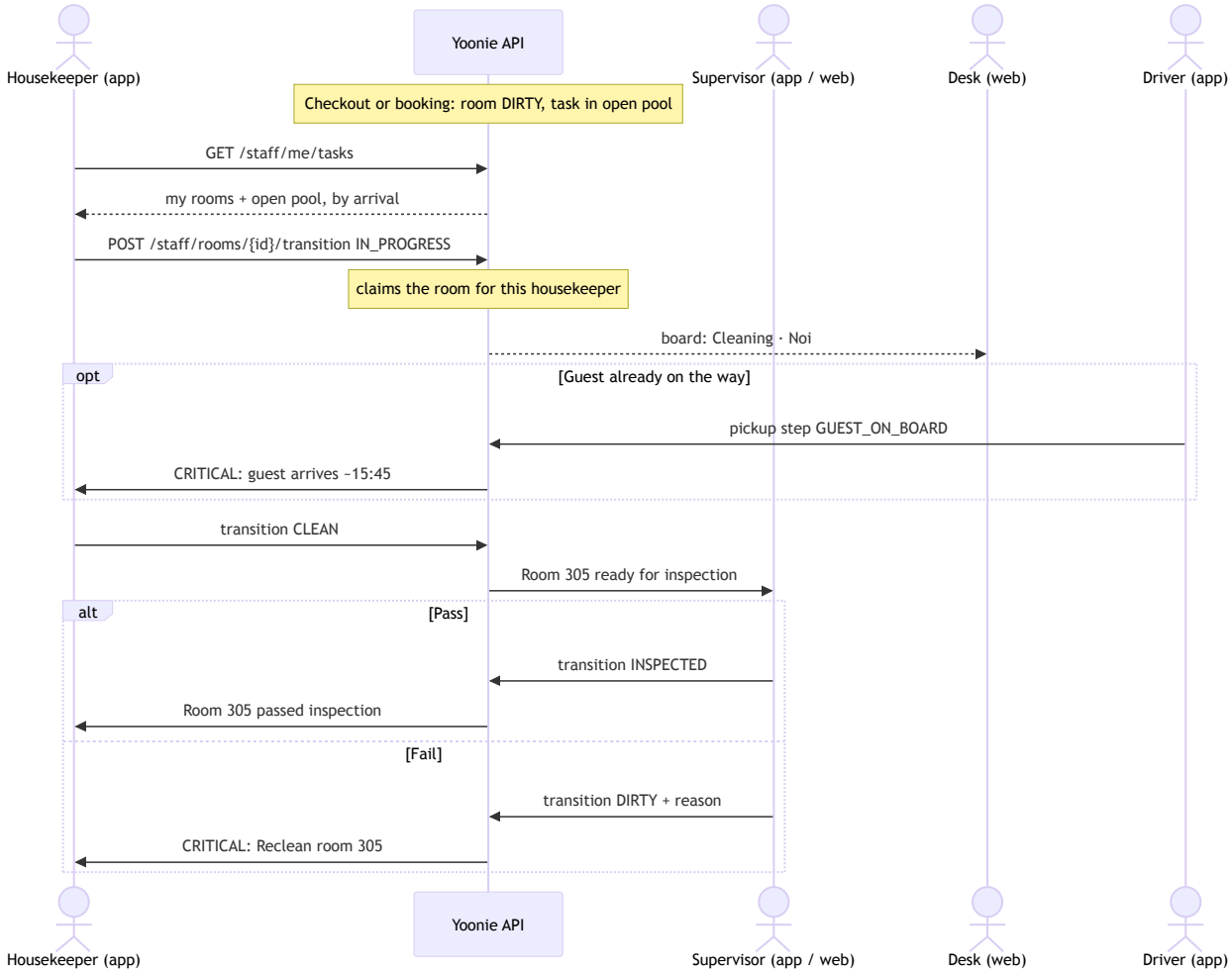
Response Body (each task)

Property Name	Description	Data Type
`_id, type, status, title`	Task identity; status OPEN, IN_PROGRESS, DONE, DISMISSED	Mixed
`assigneeUserId, assigneeName`	Owner, or empty for the open pool	String
`dueAt, priorityScore, severity`	Deadline and ordering	Mixed
`cleaningStatus`	Turnarounds: the room's live state	String
`housekeeping`	roomId, roomNumber, guestName, etaAt, targetReadyBy, etaSource, flightNumber, isVip	Object
`pickup`	guestName, flightNumber, etaAt, meetingDoor, vehiclePlate, card, stage, steps[], issue	Object
`consolidation`	serviceName, departsAt, booked, capacity, minViablePax, revenueThb, fixedCostThb, projectedLossThb, decision	Object

4 Room Turnaround

4.1 Overview

A room is inspected and ready before its guest walks in. Housekeepers take rooms from the open pool on their phones; the room and its task always move together; the supervisor inspects standing in the room.



4.2 Endpoints

Endpoint	Who	Room after	Task after
GET /staff/me/tasks	Staff, Dispatcher, Manager, Admin	–	–
POST /staff/rooms/{id}/transition IN_PROGRESS	Housekeeper, Manager	IN_PROGRESS	IN_PROGRESS, claimed
POST /staff/rooms/{id}/transition CLEAN	Housekeeper, Manager	CLEAN	DONE
POST /staff/rooms/{id}/transition INSPECTED	Manager, Admin, housekeeping:inspect	INSPECTED	DONE
POST /staff/rooms/{id}/transition DIRTY + reason	Housekeeper, Supervisor, Manager	DIRTY	OPEN (reopened)
POST /dispatch/{id}/claim	Staff, Dispatcher, Manager	–	assigned to caller
PATCH /dispatch/{id}/assign	Dispatcher, Manager, Admin	–	assigned / back to pool
GET /staff/roster?branchId	Dispatcher, Manager, Admin	–	–

4.3 Get My Tasks

Method	Request URL	Who can call
GET	`/api/staff/me/tasks`	Staff, Dispatcher, Manager, Admin

Response Body

Property Name	Description	Data Type
`housekeeping[]`	Today's rooms that are mine or unassigned, sorted by arrival	Array
`housekeeping[].cleaningStatus`	DIRTY, IN_PROGRESS, CLEAN, INSPECTED	String
`housekeeping[].targetReadyBy, etaAt`	Deadline and guest ETA	DateTime
`housekeeping[].isMine, assigneeUserId`	Mine, or open pool when empty	Mixed
`pickups[]`	My pickups today (section 5)	Array

4.4 Move a Room

Method	Request URL	Who can call
POST	`/api/staff/rooms/{id}/transition`	Staff, Manager, Admin (not Dispatcher)

Request Body

Property Name	Description	Data Type
`toStatus`	IN_PROGRESS, CLEAN, INSPECTED or DIRTY	String
`reason`	Required for DIRTY (needs cleaning / failed inspection)	String

Example

```
{
  "toStatus": "DIRTY",
  "reason": "Bathroom not done, hair in the sink"
}
```

Response Body

Property Name	Description	Data Type
`roomNumber, cleaningStatus`	The room after the move	Mixed
`isNoOp`	True when the room was already in that state (offline replay)	Boolean
`dispatchId, dispatchStatus`	The linked task after the move	Mixed
`assigneeName`	Who now holds the room's job	String
`notified[]`	Names of everyone pinged	Array

Example

```
{
  "roomId": "665a1b2c3d4e5f6a7b8c9d1a",
  "roomNumber": "305",
  "cleaningStatus": "DIRTY",
  "isNoOp": false,
  "dispatchId": "665a1b2c3d4e5f6a7b8c9d7a",
  "dispatchStatus": "OPEN",
  "assigneeName": "Noi Prasert",
  "notified": [
    "Noi Prasert"
  ]
}
```

Moves allowed: DIRTY → IN_PROGRESS → CLEAN → INSPECTED, and any → DIRTY with a reason. Anything else is **409** with the room's current status. Another housekeeper's claimed room is **403**.

4.5 Assign or Claim

Method	Request URL	Who can call
PATCH	`/api/dispatch/{id}/assign`	Dispatcher, Manager, Admin

Request Body

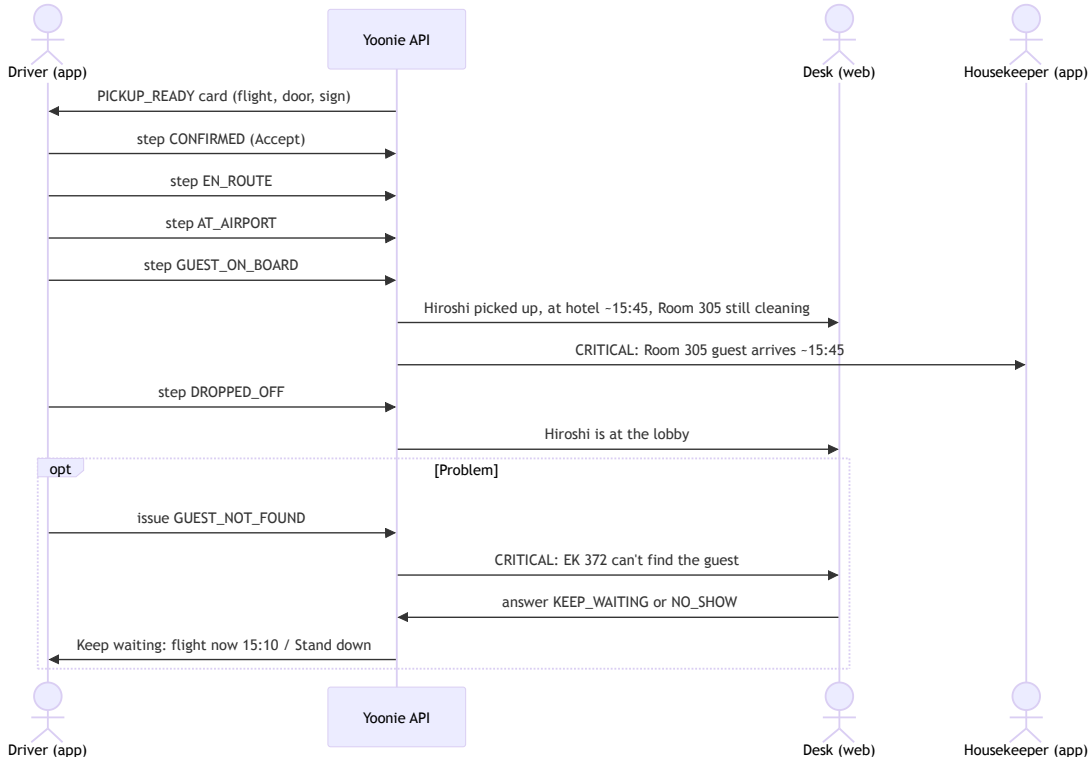
Property Name	Description	Data Type
`assigneeUserId`	Staff member on the task's branch, or null for the open pool	String | null

Response: the task plus `assigneeName` and `notified[]`. The new owner gets `TASK_ASSIGNED` (pickups: `PICKUP_READY` with the card). `POST /dispatch/{id}/claim` takes a pool task for the caller; a task someone else owns is **409**.

5 Airport Pickup

5.1 Overview

A driver is at the arrivals door with the name sign when the guest comes out, and the hotel knows when the guest will walk in. The driver reports each step from the app; the desk watches and answers problems.



5.2 Endpoints

Endpoint	Who	Stage / status after
POST /staff/pickups/{id}/step	Driver (own pickup); desk on driver's behalf	stage; status OPEN / IN_PROGRESS / DONE
POST /staff/pickups/{id}/issue	Driver; desk	issue open; severity CRITICAL
POST /dispatch/{id}/pickup-issue	Dispatcher, Manager, Admin	KEEP_WAITING: continues; NO_SHOW: DISMISSED
PATCH /orders/{id}/lines/{lineId}/flight	Dispatcher, Manager, Admin	pickup re-planned; driver must accept again
POST /staff/pickups/{id}/start, /done	Legacy app builds	EN_ROUTE, DROPPED_OFF

5.3 Record a Step

Method	Request URL	Who can call
POST	`/api/staff/pickups/{id}/step`	Staff, Dispatcher, Manager, Admin

Request Body

Property Name	Description	Data Type
`stage`	CONFIRMED, EN_ROUTE, AT_AIRPORT, GUEST_ON_BOARD, DROPPED_OFF. Forward only.	String
`note`	Optional; desk entries are prefixed "Recorded by … for the driver"	String

Example

```
{
  "stage": "GUEST_ON_BOARD"
}
```

Response Body

Property Name	Description	Data Type
`dispatchId, status`	Task after the step	Mixed
`stage`	Stage now	String
`notified[]`	Names pinged by this step	Array

Example

```
{
  "dispatchId": "665a1b2c3d4e5f6a7b8c9d7b",
  "status": "IN_PROGRESS",
  "stage": "GUEST_ON_BOARD",
  "notified": [
    "Anan Boonmee",
    "Mali Chaiyo",
    "Noi Prasert"
  ]
}
```

A repeated or earlier step is a 200 no-op. A pickup with no driver is **409**. A different driver, or a re-timed flight, sends an accepted pickup back to ASSIGNED.

5.4 Report a Problem

Method	Request URL	Who can call
POST	`/api/staff/pickups/{id}/issue`	Staff, Dispatcher, Manager, Admin

Request Body

Property Name	Description	Data Type
`kind`	FLIGHT_DELAYED, GUEST_NOT_FOUND, VEHICLE_PROBLEM, OTHER	String
`note`	What the driver sees	String

Example

```
{
  "kind": "GUEST_NOT_FOUND",
  "note": "Waited 30 min at door 4"
}
```

5.5 Answer a Problem

Method	Request URL	Who can call
POST	`/api/dispatch/{id}/pickup-issue`	Dispatcher, Manager, Admin

Request Body

Property Name	Description	Data Type
`resolution`	KEEP_WAITING or NO_SHOW	String
`note`	Instructions pushed to the driver	String

Example

```
{
  "resolution": "KEEP_WAITING",
  "note": "Flight now lands 15:10, stay at door 4"
}
```

5.6 Correct the Flight

Method	Request URL	Who can call
PATCH	`/api/orders/{id}/lines/{lineId}/flight`	Dispatcher, Manager, Admin

Request Body

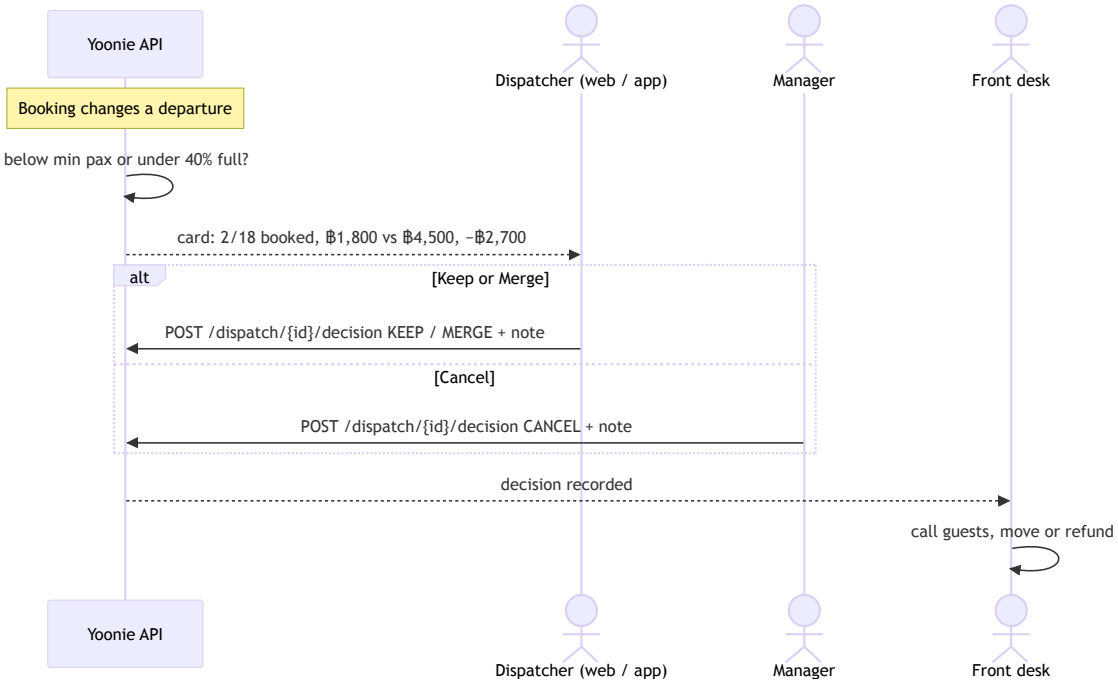
Property Name	Description	Data Type
`number`	Flight number	String
`landingAt`	Landing time (UTC)	DateTime
`meetingDoor`	Arrivals door	Number

Moves the van slot, re-plans the pickup and the room's ready-by, and pings the driver when the time changed.

6 Tour Fill Decision

6.1 Overview

A departure that would lose money is flagged before it runs. The dispatcher decides to keep or merge it; cancelling needs a manager. The front desk calls the guests afterwards.



6.2 Record a Decision

Method	Request URL	Who can call
POST	`/api/dispatch/{id}/decision`	Dispatcher (Keep, Merge), Manager and Admin (all)

Request Body

Property Name	Description	Data Type
`decision`	KEEP, MERGE or CANCEL	String
`note`	Where the guests went, or why	String

Example

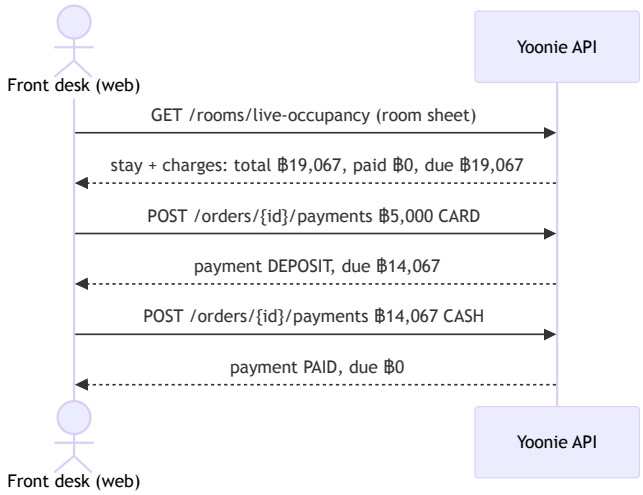
```
{
  "decision": "MERGE",
  "note": "Moved 2 guests to the 18:00 Sunday departure"
}
```

Booked	Revenue	Cost	Result	Flag
10 / 18	฿9,000	฿4,500	+฿4,500	No
5 / 18	฿4,500	฿4,500	฿0	Yes: 28% full
2 / 18	฿1,800	฿4,500	−฿2,700	Yes: below 6 guests

7 Front Desk Money

7.1 Overview

Guests pay at the hotel. The room sheet shows the folio: what was booked, service charge, VAT, what's paid and what's due. The desk records each payment; deposits are allowed, overpaying isn't.



7.2 Record a Payment

Method	Request URL	Who can call
POST	`/api/orders/{id}/payments`	Manager, Admin

Request Body

Property Name	Description	Data Type
`amountThb`	Collected now; at most the balance due	Number
`method`	CASH, CARD, PROMPTPAY, BANK_TRANSFER	String
`reference`	Slip or transfer reference	String

Example

```
{
  "amountThb": 5000,
  "method": "CARD",
  "reference": "Card slip 004512"
}
```

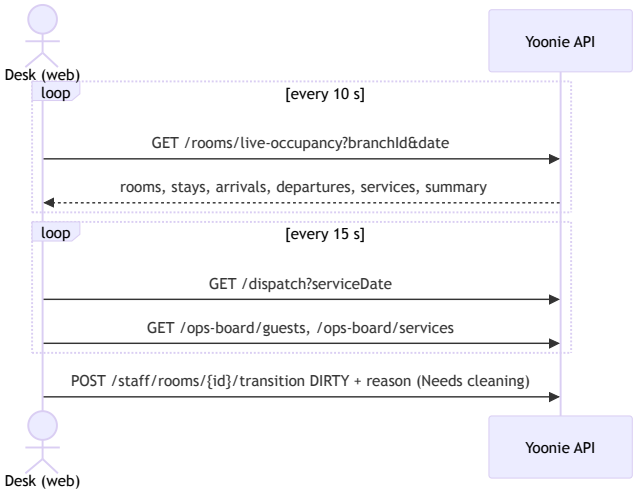
Folio (charges)

Property Name	Description	Data Type
`lines[]`	Each booked item with its total	Array
`roomsThb, servicesThb, diningThb`	Totals by kind	Number
`serviceChargeThb, vatThb`	10% and 7% (branch settings)	Number
`totalThb, paidThb, dueThb`	What's owed now	Number
`paymentStatus`	UNPAID, DEPOSIT, PAID, REFUNDED	String
`payments[]`	amountThb, method, reference, at, by	Array

8 Ops Boards

8.1 Overview

Every ops page reads live state and polls; none of them changes a room's status. Work happens in Dispatch and on phones, and shows on every board within one poll.



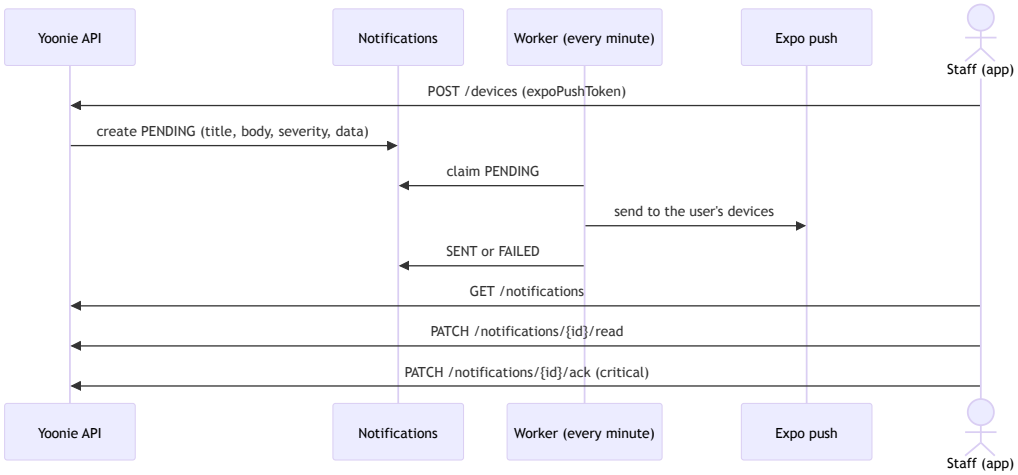
8.2 Endpoints

Endpoint	Page	Returns
GET /rooms/live-occupancy	Rooms	Every room: cleaning, occupancy, current / next stay, arrival ready-by, active task, folio
GET /ops-board/guests	Guests	In house, arriving, departing, next 7 days; services and balance
GET /ops-board/services	Services	Slots for the date: capacity, booked, revenue vs fixed cost
GET /dispatch	Dispatch	Tasks (section 3.3)
GET /staff/roster	Dispatch	Staff for assign menus: id, name, role, jobTitle

9 Notifications and Inbox

9.1 Overview

Every ping is stored first, then delivered. The worker sends pending notifications by Expo push every minute; everything is in the inbox whether or not the push arrives.



9.2 Endpoints

Endpoint	Who	Purpose
POST /devices	Any signed-in user	Register the phone's Expo push token
DELETE /devices/{id}	Owner	Stop pushes to that phone
GET /notifications	Any signed-in user	Inbox
GET /notifications/unread-count	Any signed-in user	Badge counts
PATCH /notifications/{id}/read	Owner	Mark read
PATCH /notifications/{id}/ack	Staff	Acknowledge a critical alert
POST /staff/line/link-token	Staff	Code to link a LINE account
POST /internal/notifications/tick	Scheduler	Run the worker once (cron on Vercel)

9.3 Who Gets Pinged

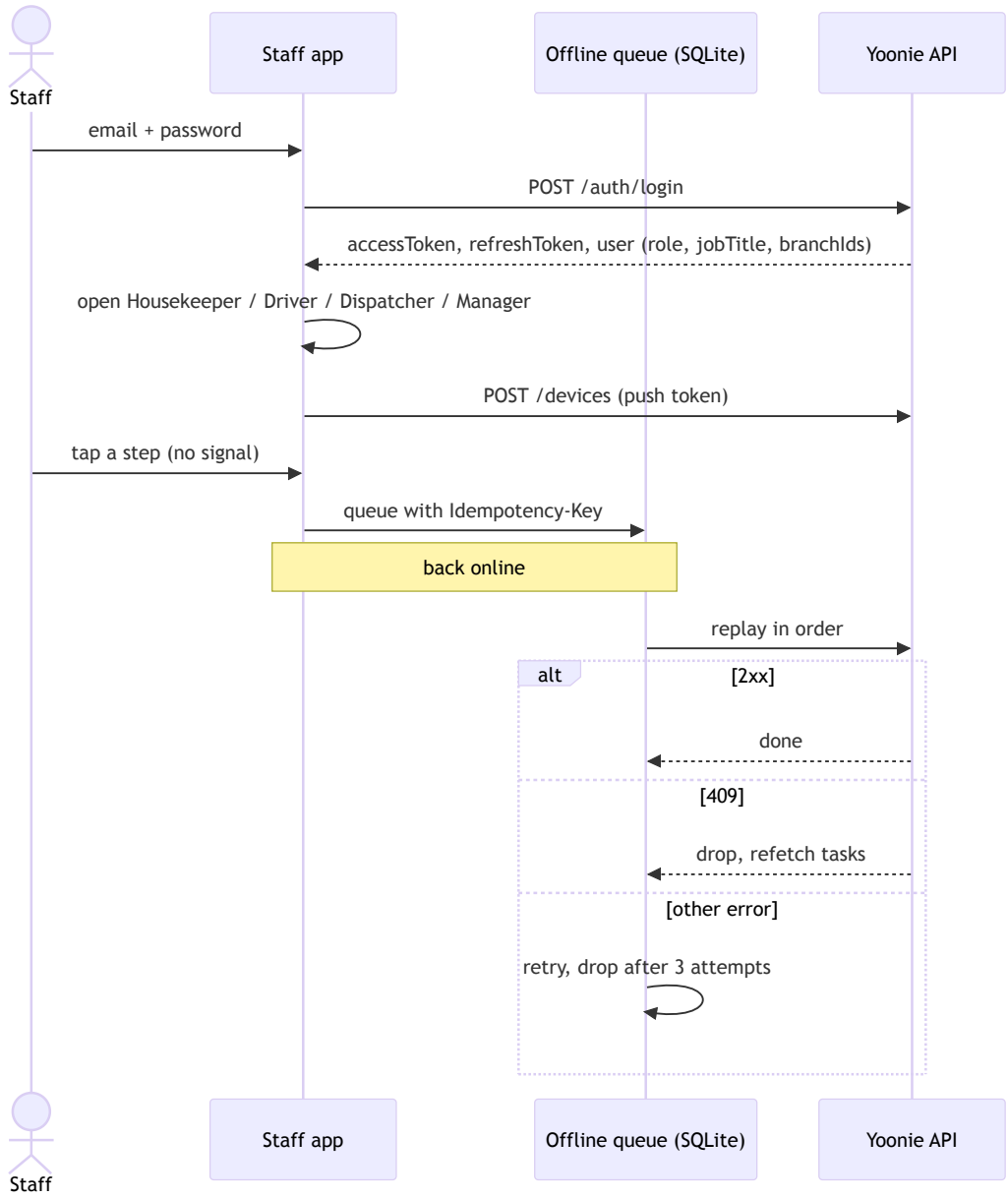
The actor is never pinged about their own action. Desk means branch managers, dispatchers, and staff whose job title is front desk, reception or concierge.

Event	Recipients	Severity
Room CLEAN	Managers, admins, housekeeping:inspect	INFO
Room DIRTY	Room's owner, or housekeeper pool	CRITICAL after CLEAN, else WARN
Room INSPECTED	Housekeeper who owned it	INFO
Room IN_PROGRESS by someone else	Owner	INFO
Task assigned	Assignee (pickup: card)	INFO / WARN
Pickup created or re-timed	Driver	INFO
Pickup GUEST_ON_BOARD	Desk; housekeeper if room not ready	INFO / WARN; CRITICAL to housekeeper
Pickup DROPPED_OFF	Desk	INFO
Pickup problem / answer	Desk / driver	CRITICAL / INFO

10 Staff App Sign-in and Offline

10.1 Overview

Staff sign in once; the app opens the screen for their job. Every action works offline: it is applied on screen straight away, queued on the phone, and replayed when the signal comes back.



10.2 Home Screen by Role

Role	Job title	Opens
STAFF	HOUSEKEEPER	Housekeeper
STAFF	DRIVER	Driver
DISPATCHER	–	Dispatcher
BRANCH_MGR, ADMIN	–	Manager (Inspect)
STAFF	anything else	"Wrong role" screen

11 Statuses

11.1 Room (cleaningStatus)

Status	Meaning	Can move to
DIRTY	Needs cleaning	IN_PROGRESS
IN_PROGRESS	Being cleaned	CLEAN, DIRTY
CLEAN	Cleaned, waiting for inspection	INSPECTED, DIRTY
INSPECTED	Ready for the guest	DIRTY

11.2 Task (status)

Status	Meaning	Can move to
OPEN	To do (owned or in the pool)	IN_PROGRESS, DONE, DISMISSED
IN_PROGRESS	Worker started	DONE, DISMISSED
DONE	Finished	OPEN (room sent back to DIRTY)
DISMISSED	No longer needed (flight moved, no-show, tour filled)	end
ACKNOWLEDGED	Retired. Old rows read as OPEN; never written	–

11.3 Pickup (stage)

Stage	Meaning	Task status
ASSIGNED	Driver given the job, not accepted yet	OPEN
CONFIRMED	Driver accepted	OPEN
EN_ROUTE	Left the hotel	IN_PROGRESS
AT_AIRPORT	At the arrivals door; the countdown stops	IN_PROGRESS
GUEST_ON_BOARD	Guest in the car	IN_PROGRESS
DROPPED_OFF	Guest at the lobby	DONE

11.4 Order, Line, Payment and Hold

Object	Statuses
Order	PENDING, CONFIRMED, CANCELLED, COMPLETED
Order line	PENDING, CONFIRMED, CANCELLED, COMPLETED, NO_SHOW
Payment	UNPAID, DEPOSIT, PAID, REFUNDED
Inventory hold	ACTIVE, CONVERTED, RELEASED, EXPIRED

12 Errors

Status	When	Client does
400	Validation failed; unknown field; DIRTY without a reason	Show the message
401	Not signed in or token expired	Refresh token, else sign in again
403	Other branch; someone else's room; role not allowed; dispatcher cancelling a tour	Show the message
404	Unknown id	Refetch
409	Move not allowed; slot full; task closed; pickup without driver; already claimed	Drop the action and refetch (app queue does this)

Status	When	Client does
429	Over 100 requests per minute from one IP	Back off

Error example


```
{
  "success": false,
  "statusCode": 409,
  "message": "Invalid room transition from CLEAN to IN_PROGRESS",
  "path": "/api/staff/rooms/665a1b2c3d4e5f6a7b8c9d1a/transition",
  "timestamp": "2026-10-25T08:12:03.120Z"
}
```

Part B · Reference

13 Data Schemas

13.1 OperationalDispatch (task)

Field	Type	Notes
branchId, serviceDate	ObjectId, YYYY-MM-DD	With type and dedupeKey: unique
type	String	HOUSEKEEPING_TURNAROUND, AIRPORT_PICKUP, TOUR_CONSOLIDATION
dedupeKey	String	hk:{room}, pickup:{order}:{line}, tour:{slot}
status, severity, priorityScore	Mixed	Section 11.2; INFO / WARN / CRITICAL
assigneeUserId	ObjectId	Owner; empty = open pool
title, dueAt, completedAt	Mixed	
housekeeping	Object	roomId, roomNumber, orderId, guestName, etaAt, etaSource, flightNumber, targetReadyBy, isVip
pickup	Object	orderId, lineId, guestName, flightNumber, etaAt, meetingDoor, vehiclePlate, driverUserId, channel, card {en, th}, stage, steps[], issue
consolidation	Object	slotKey, serviceName, departsAt, booked, capacity, minViablePax, revenueThb, fixedCostThb, projectedLossThb, decision, decisionNote

13.2 Room

Field	Type	Notes
branchId, roomNumber, floor, roomType	Mixed	Physical room. Never computes availability.
cleaningStatus, cleaningUpdatedAt	String, Date	Written only by the room transition
baseRateThb, maxOccupancy, amenities, images	Mixed	Catalogue

13.3 Order

Field	Type	Notes
orderNumber, branchId, status	Mixed	
guest	Object	fullName, email, phoneE164, lineUserId, preferredLanguage
lines[]	Array	lineId, kind, status, title, quantity, unitPriceThb, totalThb, startsAt, businessDate, slotRefs[], stay / dining / service
lines[].stay	Object	roomType, checkInDate, checkOutDate, nights, nightlyRates[], assignedRoomId, assignedRoomNumber
lines[].service	Object	serviceId, serviceKind, pax, flight {number, landingAt, meetingDoor}
totals	Object	subtotal, discount, service charge, VAT, grand total (recomputed on save)
payment	Object	method, status, entries[] {amountThb, method, reference, at, byUserId}
attribution	Object	utmSource, utmMedium, utmCampaign, channel

13.4 InventorySlot

Field	Type	Notes
slotKey	String	branch:KIND:resource:startsAt; unique
kind	String	ROOM_NIGHT, DINING, SERVICE

Field	Type	Notes
capacity, held, booked	Number	One atomic conditional $inc keeps held + booked ≤ capacity
businessDate, startsAt, endsAt	Mixed	
minViablePax, fixedCostThb	Number	Tours: feeds the fill check

13.5 Notification

Field	Type	Notes
userId, branchId	ObjectId	
type	String	TASK_ASSIGNED, ROOM_DIRTY, PICKUP_READY, CRITICAL_ALERT, GENERAL
severity	String	INFO, WARN, CRITICAL
title, body, data	Mixed	data carries dispatchId, roomId, type
status	String	PENDING, SENDING, SENT, FAILED
readAt, ackedAt	Date	

14 Roles and Permissions

Role	Can
Guest	Book, view and cancel their own order with the order token
Staff · Housekeeper	See own rooms and the pool; Start (claims), Mark clean; Needs cleaning with a reason
Staff · Driver	See own pickups; report steps and problems
Staff + housekeeping:inspect	Also inspect (Pass / Fail)
Dispatcher	See Dispatch; assign; answer pickup problems; Keep / Merge tours; record pickup steps for a driver; can't move rooms
Branch manager	Everything in their branches: room moves, inspection, Cancel tours, payments, flight edits, invite staff
Admin	As branch manager; catalogue and property setup
Platform admin	All branches; create branches

15 Decisions

#	Decision	Source
1	The worker reports work from where it happens; screens observe. Desk entries are recorded as on the worker's behalf.	Operating rules L1
2	One write path per fact: rooms move only through POST /staff/rooms/{id}/transition, which moves the room and its task together.	Operating rules L2
3	Housekeeping is an open pool: starting a room claims it; managers can assign.	Product, 25 Oct
4	ACKNOWLEDGED is retired. Ownership comes from the assignee; pickups have a real Accept step; tours have a recorded decision.	Product, 25 Oct
5	Airport pickup is a driver-reported journey with problems the desk must answer (Keep waiting / No-show).	Product, 25 Oct
6	Tour fill: dispatcher decides Keep or Merge; Cancel needs a manager.	Product, 25 Oct
7	Guest checkout confirms immediately as pay at hotel; payments are recorded at the desk.	Code
8	Room ready-by = ETA + 60 min; ETA is the flight landing, else check-in time. Guest on board tightens it to now + 45 min.	Code
9	Notifications are stored then delivered by the worker; responses return who was notified.	Operating rules L4
10	Steps only move forward; repeats are no-ops; staff mutations take an Idempotency-Key.	Operating rules L6
11	Humans decide anything that costs money or touches a guest; AI may only recommend until a rule is approved.	Operating rules L8
12	Guest tour and ops tour are separate; the ops tour only runs after login.	Product, 25 Oct

16 Open Items

Owner	Item
Product	Guides and therapists: our staff (accounts) or a partner's (no-login link)? Blocks tour departure and spa.
Product	Cut the driver to 3 taps (Accept, Guest with me, Dropped off) until "left hotel" and "at the door" feed an alert?
Product	Ready-by for check-in-time arrivals: check-in time + 60 min (15:00), or check-in time?
Product	Escalation timings (unaccepted pickup, unanswered problem, late room) as branch settings
Engineering	Ship the staff app build with pickup steps, problem reports and the dispatcher Cancel rule
Engineering	Remove web Start cleaning / Mark clean; replace with "Housekeeper phoned in…"
Engineering	Wire LINE delivery; message the guest when the driver is at the door
Engineering	Payment gateway for online payment; today everything is pay at hotel
Engineering	Airport-to-hotel drive time (45 min) as a branch setting; later measured
Ops	Vercel env: BACKEND_URL on the web project; WORKER_ENABLED on the API

17 Build Status

Item	Status	Waiting on
Guest booking (hold, order, pay at hotel)	Built	Payment gateway for online pay
Dispatch planning and inbox	Built	–
Room turnaround (pool, claim, inspect)	Built	Remove direct web buttons
Airport pickup journey (API + web)	Built	–
Airport pickup journey (staff app)	Built, not shipped	App release
Tour fill decision	Built	Guest follow-up after Merge / Cancel
Front desk payments and folio	Built	–
Ops boards (Rooms, Guests, Services)	Built	–
Push notifications and inbox	Built	WORKER_ENABLED in production
LINE delivery	Linking built; sending not wired	Engineering
Escalation timers	Designed	Branch settings
Tour departure, spa session	Designed	Who the guides and therapists are
Onboarding tours (guest, ops)	Built	–

18 Source Documents

Document	Covers	Status
docs/OPERATING_RULES.md	The laws, job types, ping matrix, escalation, AI and robots	Current: the rulebook
docs/SYSTEM_FLOWS.md	Original architecture and 05:30 run-sheet flows	Flow 4 and phases 3–4 outdated
docs/DATA_MODEL.md	ERD and state machines	Room and task lifecycles superseded by section 11
docs/FRONTEND_API.md	Endpoint guide	Predates Dispatch claim / assign / pickup steps
docs/MULTI_TENANCY.md	Hosts, branches, proxy routing	Current
PRODUCT.md	Pitch to code map, product principles	Current
Swagger /api/docs	Every endpoint and DTO	Current: generated from code